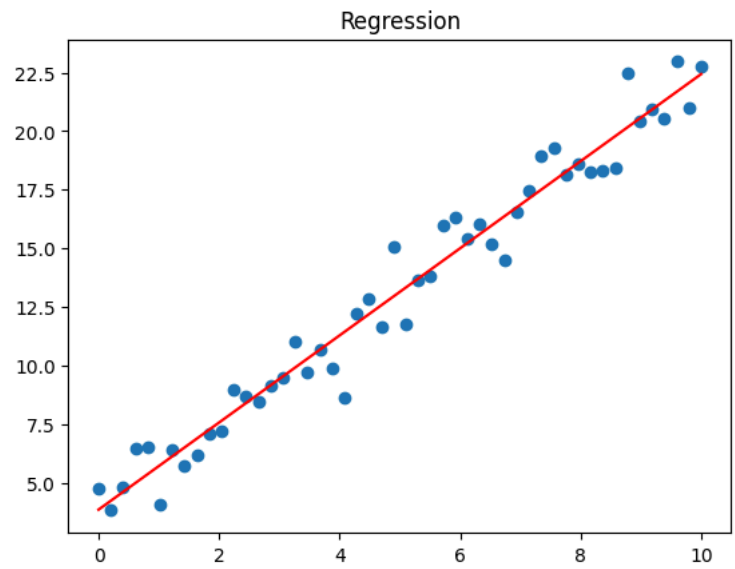
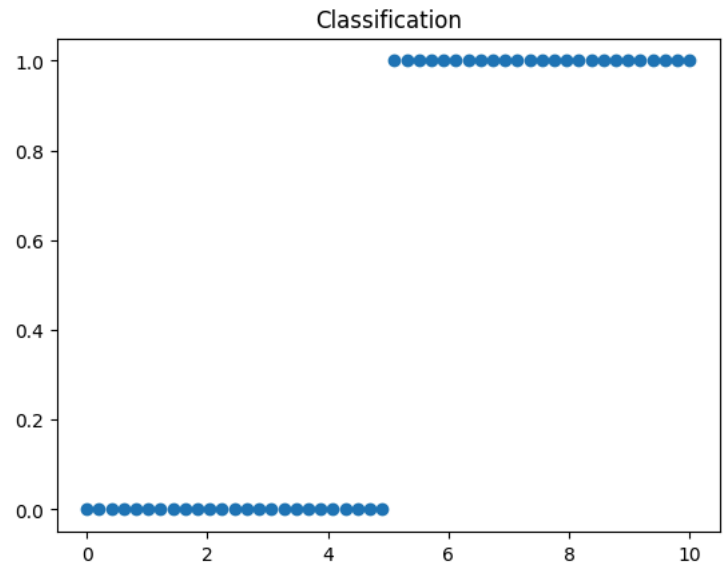
```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(0)
X_reg = np.linspace(0, 10, 50)
y_reg = 2 * X_reg + 3 + np.random.randn(50)
X_clf = np.linspace(0, 10, 50)
y_clf = np.where(X_clf > 5, 1, 0)
```

```
coeffs = np.polyfit(X_reg, y_reg, 1)
y_pred = np.polyval(coeffs, X_reg)

plt.scatter(X_reg, y_reg)
plt.plot(X_reg, y_pred, color='red')
plt.title("Regression")
plt.show()
```




```
threshold = 5
y_pred_clf = np.where(X_clf > threshold, 1, 0)

plt.scatter(X_clf, y_clf)
plt.title("Classification")
plt.show()
```




```
m = 2
c = 3

y_pred_manual = m * X_reg + c
y_pred_manual
```

```
array([ 3.        ,  3.40816327,  3.81632653,  4.2244898 ,  4.63265306,
        5.04081633,  5.44897959,  5.85714286,  6.26530612,  6.67346939,
        7.08163265,  7.48979592,  7.89795918,  8.30612245,  8.71428571,
        9.12244898,  9.53061224,  9.93877551, 10.34693878, 10.75510204,
       11.16326531, 11.57142857, 11.97959184, 12.3877551 , 12.79591837,
       13.20408163, 13.6122449 , 14.02040816, 14.42857143, 14.83673469,
       15.24489796, 15.65306122, 16.06122449, 16.46938776, 16.87755102,
       17.28571429, 17.69387755, 18.10204082, 18.51020408, 18.91836735,
       19.32653061, 19.73469388, 20.14285714, 20.55102041, 20.95918367,
       21.36734694, 21.7755102 , 22.18367347, 22.59183673, 23.        ])
```

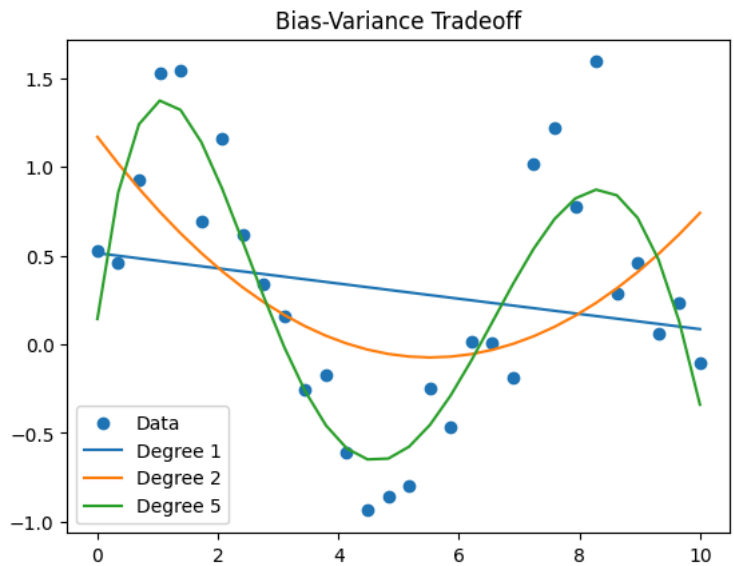
```
mse = np.mean((y_reg - y_pred_manual) ** 2)
print("MSE:", mse)
```

```
MSE: 1.2865587537274976
```

```
threshold = 5
y_pred = np.where(X_clf > threshold, 1, 0)

accuracy = np.mean(y_pred == y_clf)
print("Accuracy:", accuracy)
```

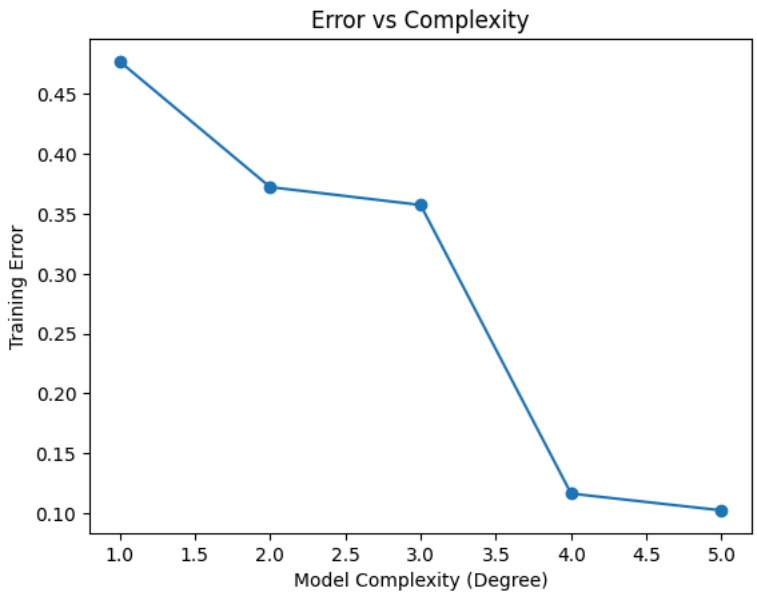
```
Accuracy: 1.0
```

```
import numpy as np
import matplotlib.pyplot as plt


np.random.seed(0)
X = np.linspace(0, 10, 30)
y = np.sin(X) + np.random.randn(30) * 0.3


def fit_model(degree):
    coeffs = np.polyfit(X, y, degree)
    y_pred = np.polyval(coeffs, X)
    return y_pred

y_pred_1 = fit_model(1)
y_pred_2 = fit_model(2)
y_pred_5 = fit_model(5)

plt.scatter(X, y, label="Data")

plt.plot(X, y_pred_1, label="Degree 1")
plt.plot(X, y_pred_2, label="Degree 2")
plt.plot(X, y_pred_5, label="Degree 5")

plt.legend()
plt.title("Bias-Variance Tradeoff")
plt.show()
```




```
degrees = [1, 2, 3, 4, 5]
errors = []

for d in degrees:
    coeffs = np.polyfit(X, y, d)
```

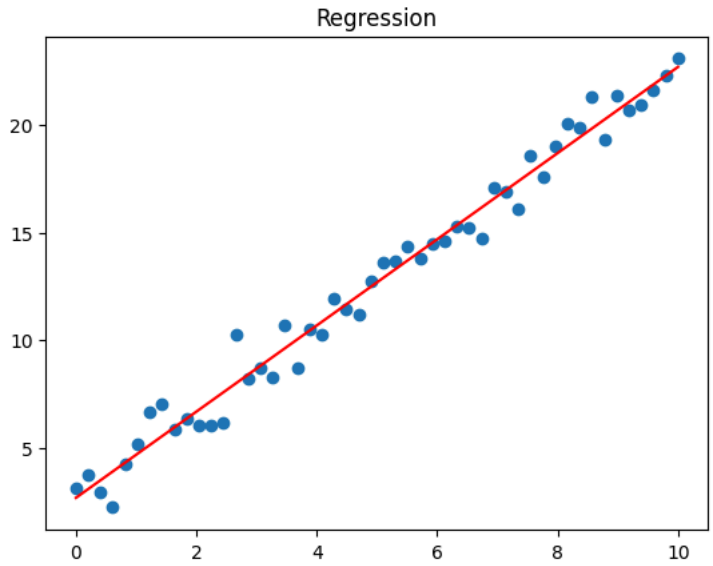
```
    y_pred = np.polyval(coeffs, X)
    mse = np.mean((y - y_pred) ** 2)
    errors.append(mse)

plt.plot(degrees, errors, marker='o')
plt.xlabel("Model Complexity (Degree)")
plt.ylabel("Training Error")
plt.title("Error vs Complexity")
plt.show()
```



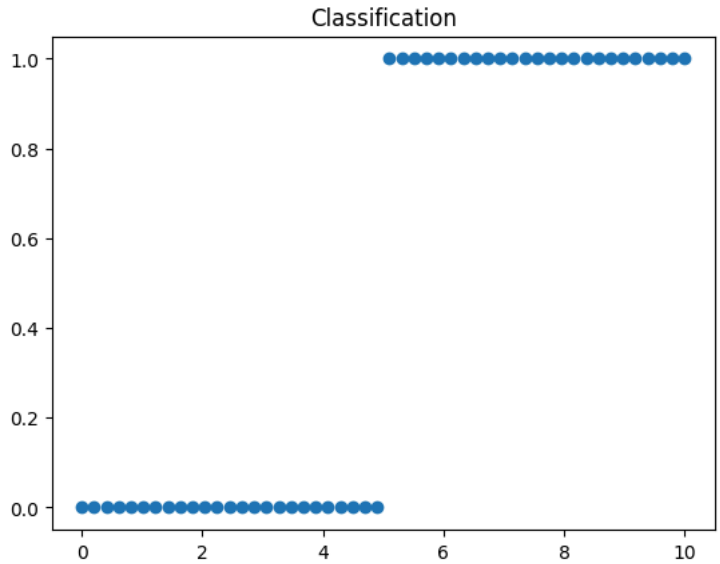

```
import numpy as np

def calculate_mse(y_true, y_pred):
    return np.mean((y_true - y_pred) ** 2)

y_true = np.array([10, 20, 30])
y_pred = np.array([12, 18, 33])

print(calculate_mse(y_true, y_pred))
```

```
5.666666666666667
```

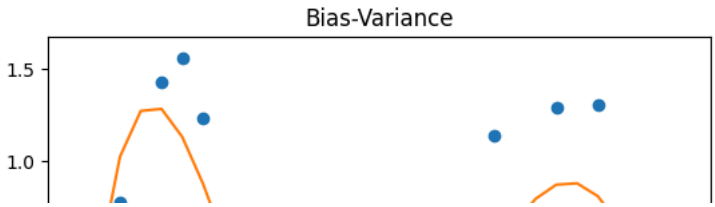
```
import numpy as np
import matplotlib.pyplot as plt
X = np.linspace(0, 10, 50)
y = 2 * X + 3 + np.random.randn(50)

coeffs = np.polyfit(X, y, 1)
y_pred = np.polyval(coeffs, X)

plt.scatter(X, y)
plt.plot(X, y_pred, color='red')
plt.title("Regression")
plt.show()
```




```
X = np.linspace(0, 10, 50)
y = np.where(X > 5, 1, 0)

plt.scatter(X, y)
plt.title("Classification")
plt.show()
```




```
X = np.linspace(0, 10, 30)
y = np.sin(X) + np.random.randn(30)*0.3

y1 = np.polyval(np.polyfit(X, y, 1), X)
y5 = np.polyval(np.polyfit(X, y, 5), X)

plt.scatter(X, y)
plt.plot(X, y1, label="Underfit")
plt.plot(X, y5, label="Overfit")
plt.legend()
plt.title("Bias-Variance")
plt.show()
```


Bias-Variance
1.5
1.0